ASSIGNMENT 17
SQL Injection in an Android Login Application

1. Introduction

SQL Injection is a security vulnerability that occurs when an application combines untrusted input with an SQL query without safely separating data from commands. In a vulnerable Android login application, an attacker may alter the meaning of a database query and cause the application to accept a login without valid credentials. This practical demonstrates the concept using a local, intentionally vulnerable example and explains how to prevent authentication bypass.

2. Objectives

- Understand SQL Injection in the context of an Android login screen.
- Observe how unsafe query construction can change an authentication condition.
- Explain why authentication bypass may occur.
- Recommend secure query handling and authentication practices.

3. Requirements

- Android Studio or another suitable Android development environment.
- A local demonstration app created for the practical, with a test-only database and dummy account.
- SQLite or an equivalent local test database.
- An Android emulator or test device.
- A record sheet to document the expected and observed behavior.

Perform the demonstration only in an application and database you own or are authorized to test. Do not test a real service or use real credentials.

4. Theory

A login system commonly checks whether a username and password match a stored record. If the application builds the SQL statement by directly joining user-entered strings into the query, quotation marks and SQL operators supplied as input may change the query's structure. The database may then evaluate a condition the developer did not intend.

A vulnerable implementation may construct a statement by concatenating the username and password directly into the SQL text. This is unsafe because input may be interpreted as part of the SQL command instead of as data.

A secure implementation uses parameterized queries. The database receives the SQL structure separately from the input values, preventing input from changing the intended query logic.

5. Procedure

Step 1: Set up a local test application

Create or use a deliberately vulnerable login app in an isolated emulator. Populate its local database with dummy data only, such as a test account named student with a non-sensitive test password. Do not include real accounts or production data.

Step 2: Review the login query

Locate the method that reads the username and password fields. Check whether it inserts those values directly into an SQL string. Record the query-building approach and determine whether input values are bound as parameters.

Step 3: Establish normal login behavior

Enter the valid dummy credentials and submit the form. Record whether the application grants access. Then enter an incorrect password and record whether the application rejects the login. These baseline checks help compare normal behavior with the intentionally vulnerable case.

Step 4: Demonstrate the injection concept safely

In the isolated lab, use a purpose-built test case or instructor-provided demonstration input that changes a SQL condition in the local dummy database. Do not use this technique against real applications. Observe whether the vulnerable implementation returns an unexpected matching record or treats the login as successful.

The key observation is that unsafe concatenation lets input become part of the SQL command. If the application decides login success merely from whether the altered query returns a row, the attacker may bypass the intended credential check.

Step 5: Analyze the authentication bypass

- User input is concatenated into the SQL statement rather than bound as data.
- The resulting query may have a different logical condition than the developer intended.
- The database may return a record even though the submitted password is not valid.
- The application treats the returned record as proof of authentication and grants access.
- This can expose account data or protected screens, depending on what the application permits after login.

Step 6: Fix and retest

Replace dynamic string concatenation with a parameterized query. With SQLiteDatabase, use selection arguments rather than inserting user input into the SQL string. For example:

```
Cursor c = db.query("users", null, "username = ? AND password_hash = ?",
    new String[] { username, passwordHash }, null, null, null);
```

This example shows parameter binding; a real application should also use an appropriate password-hashing design and avoid storing plaintext passwords. Check that the result contains the expected account record, close the cursor, and reject login when no matching record is found. Repeat the

normal and invalid-login tests, and confirm that the injection test case no longer bypasses authentication.

6. Analysis

Authentication bypass occurs when the application trusts the result of a manipulated SQL query instead of verifying the submitted credentials correctly. Parameterized queries prevent SQL syntax in input from changing the structure of the statement. Additional protections include hashing passwords with a suitable password-hashing algorithm, minimizing stored data, applying server-side authorization where applicable, and avoiding detailed database errors in user-facing messages.

7. Observation Table

Test case	Expected behavior	Vulnerable behavior	Conclusion
Valid dummy credentials	Login succeeds	Login succeeds	Normal behavior
Incorrect password	Login rejected	Login rejected in a basic test	Baseline negative test
SQL condition-altering test input in local lab	Login rejected	May return a row and allow login	Potential authentication bypass
Same test after parameterized-query fix	Login rejected	Login rejected	Injection blocked by parameter binding

8. Result

The practical explains how unsafe SQL query construction in a deliberately vulnerable Android login application can alter authentication logic and potentially allow login without valid credentials. Parameterized queries and secure password handling are recommended to prevent this weakness. The observation table presents illustrative expected outcomes; actual results should be recorded from the authorized local lab.

9. Precautions

- Use only a deliberately vulnerable test app and dummy database records.
- Do not test public, production, or third-party applications without explicit written authorization.
- Never use real passwords or sensitive user information in the lab.
- Do not store plaintext passwords; use appropriate password hashing.
- Retest after remediation to confirm that normal login still works and invalid login attempts are rejected.

10. Conclusion

SQL Injection can cause authentication bypass when untrusted input changes the structure of an SQL query and the application treats the resulting database record as successful authentication. Using parameterized queries, secure password storage, and proper authorization checks significantly reduces this risk. Testing should remain confined to an authorized local environment.

11. References

- OWASP SQL Injection Prevention Cheat Sheet.
- OWASP Mobile Application Security Testing Guide (MASTG).
- Android Developers documentation for SQLiteDatabase and parameterized queries.